

PROGRAM FLOW

Control flow is the order in which the set of instructions (program) are executed or evaluated. Based on how expressions evaluate, a program can decide to skip instructions, repeat them, or choose one of several instructions to run. Infact, you almost never want your programs to start from the first line of code and simply execute every line, straight to the end. Control flow statements decide the Python instructions to execute under certain conditions.

COMPARISON OPERATORS

Comparison operators compare two values and they evaluate to a Boolean value (True or False) depending on the values being compared.

OPERATOR	NAME	DESCRIPTION	EXAMPLE
==	Equal	Returns True if both values are equal, otherwise returns False	x == 3
!=	Not equal	Returns True if both values are not equal, otherwise returns False	x != 3
>	Greater than	Returns True if the value on the left is greater than the value on the right, otherwise returns False	x > 3
<	Less than	Returns True if the value on the left is less than the value on the right, otherwise returns False	x < 3
>=	Greater than or equal to	Returns True if the value on the left is greater than or equal to the value on the right, otherwise returns False	x >= 3
<=	Less than or equal to	Returns True if the value on the left is less than or equal to the value on the right, otherwise returns False	x <= 3

```
In [ ]: age = 10
age > 18
```

```
In [ ]: 2 == 2
```

```
In [ ]: 'age' == 'year'
```

```
In [ ]: 'age' == 'Age'
```

BOOLEAN OPERATORS

Boolean operators are used to compare Boolean values. There are three Boolean operators in Python (**and**, **or** and **not**) and they evaluate to a Boolean value. The **and** and **or** operators are considered **binary operators** because they always operate on two Boolean values or Boolean expressions. The **not** operator is considered a **unary operator** as it operates on only one Boolean value or expression.

The **and** operator evaluates to **True** if and only if both Boolean values are **True** or both Boolean expressions evaluate to **True**. Let's look at the **Truth Table** for the **and** operator which shows every possible result for the **and** Boolean operator.

EXPRESSION	RESULT	EXAMPLE
True and True	True	(5 > 3) and (0 < 5)
True and False	False	(5 > 3) and (0 < -5)
False and True	False	(5 > 30) and (0 < 5)
False and False	False	(5 > 30) and (0 < -5)

```
In [ ]: (5 > 3) and (0 < 5)
```

```
In [ ]: (5 > 3) and (0 < -5)
```

The **or** operator evaluates to **True** if and only if either of the Boolean values is **True** or either of the Boolean expressions evaluates to **True**. If both values are **False**, the **or** operator evaluates to **False**. Let's look at the **Truth Table** for the **or** operator which shows every possible result for the **or** Boolean operator.

EXPRESSION	RESULT	EXAMPLE
True or True	True	(5 > 3) or (0 < 20)
True or False	True	(5 > 3) or (0 < -20)
False or True	True	(5 > 30) or (0 < 20)
False or False	False	(5 > 30) or (0 < -20)

```
In [ ]: (5 > 3) or (0 < 20)
```

```
In [ ]: (5 > 30) or (0 < -20)
```

The **not** operator evaluates to the opposite of the Boolean value. Below is the **Truth Table** for the **not** Boolean operator.

EXPRESSION	RESULT	EXAMPLE
not True	False	not (3 > 2)
not False	True	not (3 > 20)

```
In [ ]: not(3 > 2)
```

```
In [ ]: not (3 > 20)
```

CONTROL FLOW STATEMENTS

Control flow statements are made up of two parts; the **condition** and the **block of code**.

The **condition** is a **Boolean expression** that evaluates to True or False and based on this condition, a control flow statement decides what to do.

The **Block of code** is a group of code, usually indented, to be executed based on the condition.

IF STATEMENT

The **if** statement is the most common type of control flow statement. The **block of code** immediately after the **if** statement is executed if the **condition** is **True** otherwise, it is skipped. Below is an example of the **if** statement.

```
In [1]: age = 27
if age >= 18:
    print('You are an adult')

You are an adult
```

IF / ELSE STATEMENT

An **if / else** statement gives us two possibilities. The block of code immediately after the **else** statement is executed if the **condition** is **False**.

```
In [ ]: age = 16
if age >= 18:
    print('You are an adult.')
else:
    print('You are young.')
```

We can have a **nested if** or **if / else** statements. Let's see an example.

```
In [15]: a=1
b=2
c=3
if(a>b):
    print('a is greater')
if(b>a>c):
    print('b is greater')
else:
    print('c is greater')

c is greater
```

```
In [17]: a=1
if(a>0):
    print('a is postive')
```

a is postive

```
In [7]: a = -5
if a >= 0:
    if (a % 2 == 0):
        print(a, 'is even.')
    else:
        print(a, 'is odd.')
else:
    print( f'{a}, is a negative.')
```

-5, is a negative.

IF / ELIF STATEMENT

Unlike the **if / else** statement, the **if / elif** gives us more than two possibilities. The **elif** simply means 'else if' and it immediately follows an **if** or another **elif** statement. We can also have an **else** statement with an **if/elif** statement.

```
In [28]: score = 55
if (score < 40):
    grade = 'FAIL'
elif (score >= 40 and score < 55):
    grade = 'PASS'
elif (score >= 55 and score < 70):
    grade = 'MERIT'
elif (score >= 70):
    grade = 'DISTINCTION'
print('GRADE:',grade)
```

GRADE: MERIT

```
In [38]: score=20
if(score<40):
    grad= 'F9'
elif(score==40 and score<45):
    grad= 'B8'
elif(score==45 and score<50):
    grad= 'D7'
elif(score==50 and score<55):
    grad= 'C6'
elif(score==55 and score<59):
    grad= 'C5'
elif(score==60 and score<64):
    grad= 'C4'
elif(score==65 and score<69):
    grad= 'B3'
elif(score==70 and score<75):
    grad= 'B2'
elif(score==75):
    grad= 'A1'
print(grad)

F9
```

The above program checks the value stored in the score variable and assigns a value to the grade variable based on range of the score value.

LOOPS

Loop is one of the major tool of control flow that allows us to execute a block of code many times as long as the loop **condition** is **True**. There are two major types of loop:

- while loop
- for loop

WHILE LOOP

Generally, the **while** loop is used to keep looping through a block of code while the condition is True. It can also be used to loop through a block of code a certain number of times as seen in the code below.

```
In [39]: x = 5
while(x <= 10):
    print(x)
    x += 1

5
6
7
8
9
10
```

As seen in the example above, In a while loop, we need to have initialised a variable before the while statement. In the case of our example, 'a' was initialised to 5. Another important thing to note is that the variable must be incremented to prevent the loop from executing forever (infinite loop).

An **infinite loop** is a loop that never terminates. It is useful in client / server programming so that servers can continually communicate or receive requests from its clients.

THE BREAK STATEMENT

The **break** statement is used to **exit** a loop when the loop condition is still **True**.

```
In [40]: my_number = 5
while True:
    number = int(input('Guess a number:'))
    if number == my_number:
        print('Yay!! Your guess was right.')
        break
```

Guess a number:5
Yay!! Your guess was right.

THE CONTINUE STATEMENT

The **continue** statement is used to stop the current iteration and move on to the next iteration. When the program execution reaches a continue statement, the program execution immediately jumps back to the start of the loop and reevaluates the loop's condition. Suppose we want to print odd numbers between 1 and 50, we can use a continue statement to achieve it.

```
In [41]: a = 0
while (a < 50):
    a += 1
    if(a % 2 == 0):
        continue
    print(a)
```

1
3
5
7
9
11
13
15
17
19
21
23
25
27
29
31
33
35
37
39
41
43
45
47
49

TRUTHY AND FALSY VALUES

In Python and some other programming languages, values evaluate to either **True** or **False**. As the name implies, **Truthy** values are values that evaluate to **True** and **Falsy** values are values that evaluate to **False**. **Falsy values** are 0, 0.0, "" (the empty string), any empty data structure, None and False. Other values are **Truthy values**.

We can use falsy or truthy values in an **if** or **while** conditional statement. let's see an example.

```
In [42]: name = ''
while (not name):
    name = input('What is your name?')
print(name)
```

What is your name?azeez
azeez

In the code above, the name variable contains an empty string which is a falsy value and a falsy value would evaluate to false. So the condition `not name` would evaluate to True. Hence the while block of code would be executed.

```
In [43]: num = 0
if num:
    print('Number is', num)
```

In the code above the num variable is assigned a value of 0 and 0 is a falsy value. So it means the condition num would evaluate to false. Hence, the **if** block of code will not be executed. If we change the value of num to an integer value other than 0, the **if** block of code would be executed because all integer number other than zero are truthy values which evaluates to True.

THE ELSE STATEMENT

The **else** statement can also be used with **loops**. The else part is executed when the condition is no longer **True**.

```
In [44]: a = 1
while (a < 6):
    print (a)
    a += 1
else:
    print ('a is no longer less than 6')
```

1
2
3
4
5
a is no longer less than 6

```
In [ ]: a = 20
while (a < 27):
    a += 1
    if(a % 2 == 0):
        continue
    print(a)
```

```
In [48]: a = 20
while (a < 27):
    a+=1
    if(a % 2 == 1):
        continue
    print(a)
```

22
24
26

```
In [54]: a=1
while(a<=10):
    print(a)
    a += 1
```

1
2
3
4
5
6
7
8
9
10

FOR LOOPS

For loops are used for **iterating** over a **sequence** (e.g. strings, lists, dictionaries, tuples, lines in a file etc). We can use **for** loop and the **range()** function to loop through a block of code a certain number of times.

```
In [55]: name = 'Brian'
for c in name:
    print(c)
print('End of loop')
```

B
r
i
a
n
End of loop

In the above example, we used the **for** loop to iterate through the string and to print each cahracter in the string. As used in the code above,'character' is just a variable name, we could use any variable name like a, letter, alphabet etc. We can also use **for** loops to iterate through a list. We would discuss lists and other data structures later in the course.

```
In [56]: colors = ['red', 'blue', 'green', 'yellow']
for color in colors:
    print (color)
```

red
blue
green
yellow

Let's see an example of using the **range()** function with **for** loop to print even numbers between 1 and 15.

```
In [57]: for i in range(1, 15):
        if ( i % 2 == 0):
            print(i)
```

2
4
6
8
10
12
14

```
In [ ]: for a in range(10, 2, -1):
        print(a)
```

Another way of writing the above code using the **range()** function is:

```
In [ ]: for i in range(2,15,2):
        print(i)
```

Just like **while** loops, **break**, **continue** and **else** statements can be used in **for** loops.

```
In [ ]: fruits = ['banana', 'mango', 'apple', 'orange', 'grape']
for fruit in fruits:
    if fruit == 'apple':
        continue
    print(fruit)
```

EXERCISE: GUESS THE NUMBER

In this exercise, initialise a **variable** with number between **1** and **20**. Giving the user **6 chances**, tell the user to **guess your chosen number**. Tell the user if his/her **chosen number** is **greater** or **lesser than** your **chosen number** or if he/she **guessed your chosen number right**.

```
In [94]: num=2
myNum=int(input('Guess my chosen number'))
i = 0
while True:

    print('You Guessed the number right')
    break
if(myNum>2):
    print('Your Guessed number is greater')
if(myNum<2):
    print('Your Guessed number is less')
print(myNum)
```

Guess my chosen number6
You Guessed the number right
Your Guessed number is greater
6

```
In [ ]:
```